SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

CO3 ASSESSMENT TOOL 2 – Code Review & Repository Evaluation

Course Code:	CSA10 / CSA1025	**Course Title:**	Software Engineering
CO Assessed:	CO3	**Assessment:**	Assessment Tool 2 – Code Review & Repository Evaluation
Weightage:	20%	**Total Marks:**	25
CO3 Statement:	Build full-stack applications with an emphasis on containerization, version control, and collaborative development		
Problem Statement:	**Bus Ticket Reservation System**		
Student Name:	K. Sampath Reddy	**Reg. No.:**	192424285
Date:	20-10-2026	**Duration:**	60 minutes

Code of Conduct

I **K Sampath Reddy (192424285)** *certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the Student: _____**K Sampath Reddy**_____

Annexure A – Code Review & Repository Evaluation

BusBuddy: Code Review and Repository Evaluation

Review of source code quality, repository organization, Git practice, testing and documentation – with evidence, findings and a prioritized improvement plan

No.	Section (Annexure A)	No.	Section (Annexure A)
1	Problem Overview	5	Code Testing and Validation
2	Repository Organization	6	Repository Documentation
3	Code Quality Review	7	Code Review Findings and Recommendations
4	Version Control Evaluation	–	Deliverables Checklist and References

Review at a Glance

4.5 / 5	90 %	15	7
Version control practice	Line coverage · 6 of 6 tests pass	Test cases run · 12 pass · 3 fail	Findings · 2 high · 2 medium · 3 low

Verdict: BusBuddy is a small, readable and well-modularized project with an unusually clean Git history. Its main weakness is **input validation and error handling**: three edge cases exposed real defects, one of which crashes the server. All seven findings have concrete fixes in Section 7.

1. Problem Overview

1.1 Assigned Problem and Implemented Solution

The assigned problem statement is the **Bus Ticket Reservation System**. Passengers must be able to search for a bus, see which seats are really free, book a seat and cancel it for a refund. The hard part is that many people may try to buy the **same seat at the same moment**, so the system must never sell one seat twice.

The implemented solution, **BusBuddy**, is a Node.js REST API with a small web page. It adds four ideas that make it different from a basic booking form: an **8-minute seat lock** while the passenger pays, **double-booking protection** in two layers (Redis lock and a database rule), a **live seat map** (available / held / booked) and an **automatic slab-based refund**. The whole system starts with one command (`docker compose up --build`).

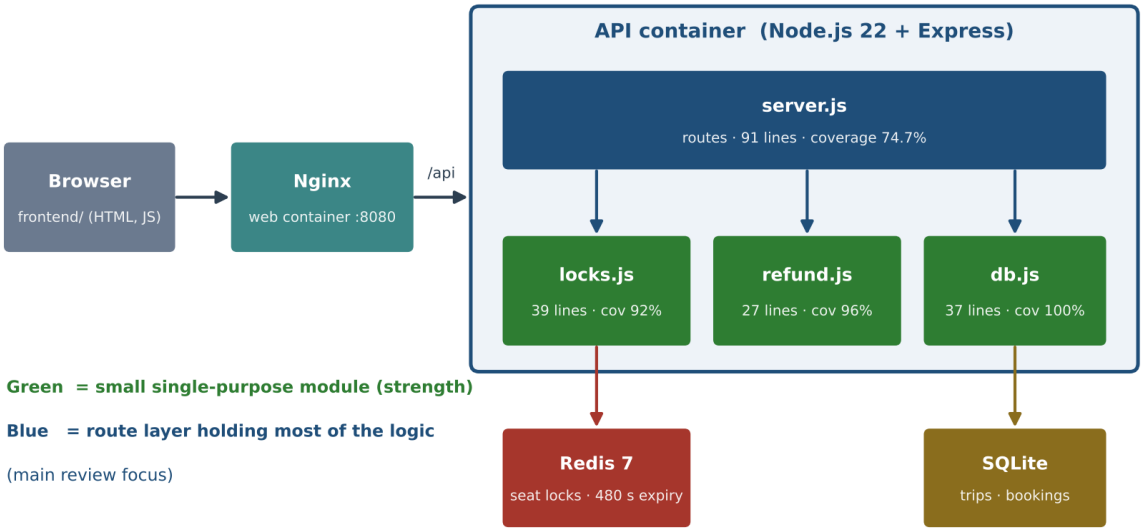


Figure 1: *System architecture of BusBuddy with the size and test coverage of each code module (measured in this review).*

1.2 Project Objectives

ID	Objective	How the review checked it
O-1	Never sell one seat twice	Second lock and parallel locks rejected (HTTP 409); database UNIQUE rule – test cases TC-04, TC-12
O-2	Hold a seat for a short, fixed time	Redis key expires after 480 seconds – TC-05 (redis-cli ttl)
O-3	Refund automatically by slab	90 % / 50 % / 25 % / 0 % – TC-08, TC-11 and unit tests
O-4	Keep every change traceable	34 commits with typed messages, 2 release tags – Section 4
O-5	Run the same way on every machine	Dockerfile + Compose stack – Sections 2 and 6

1.3 Key Functionalities

ID	Functionality	Implemented in	API call
FR-A	Search trips by source and destination	server.js	GET /api/trips
FR-B	Show live seat status	server.js, locks.js	GET /api/trips/:id/seats
FR-C	Hold seats for 8 minutes	locks.js	POST /api/trips/:id/lock
FR-D	Confirm a booking only for a locked seat	server.js, db.js	POST /api/bookings
FR-E	Cancel a booking and calculate the refund	refund.js	DELETE /api/bookings/:id

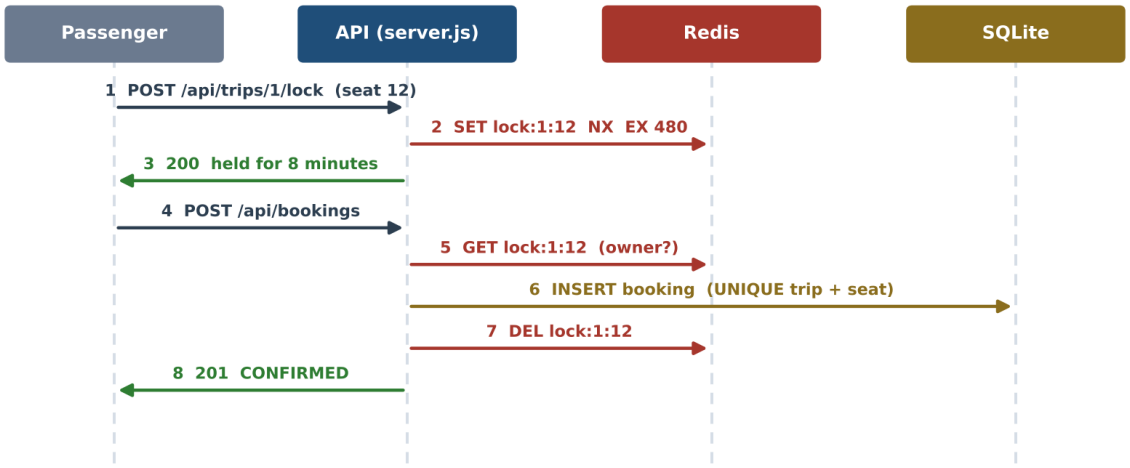


Figure 2: *Request flow for holding a seat and confirming a booking (FR-C and FR-D).*

1.4 Scope and Method of This Review

Review activity	What was done
Static reading	All 4 source files and 3 test files (276 lines) were read line by line against a checklist: readability, modularity, standards, documentation, error handling, security, testability.
Repository analysis	Folder tree, naming, .gitignore, commit history (git log --graph), branches, tags and merge commits were inspected.
Automated tests	npm test (Node built-in test runner) with coverage (--experimental-test-coverage).
Live API probing	The API was run against a real Redis server; normal, concurrent and invalid requests were sent with curl.
Documentation check	README, changelog, workflow guide, .env.example and CI file were compared against a documentation checklist.

2. Repository Organization

The repository separates the three parts of the system – API, web page and reverse proxy – so each can be changed, tested and containerized on its own. Figure 3 shows the real folder tree (8 directories, 22 files) taken from the repository.



Figure 3: *Folder structure (output of `tree`).*

Repository layout scorecard

Aspect	Rating	Evidence
Separation of concerns	**Strong**	backend/, frontend/, nginx/ are independent.
Source vs tests	**Strong**	Code in src/, tests in test/.
Config kept out of code	**Strong**	.env.example committed; real .env ignored.
Container files beside code	**Strong**	Dockerfile and .dockerignore live in backend/.
Automation	**Good**	CI file in .github/workflows/.
Docs folder	**Fair**	Only GIT_WORKFLOW.md; no API reference.

2.1 Naming Conventions

Item	Convention used	Assessment
Folders	lowercase, one word (backend, frontend, docs)	Consistent and predictable.
Source files	lowercase nouns (locks.js, refund.js, db.js)	File name states its single job.
Test files	<module>.test.js, helper fakeRedis.js	Matches the glob test/*.test.js.
Root documents	UPPERCASE (README.md, CHANGELOG.md)	Standard open-source convention.
Branches	feature/seat-lock, hotfix/refund-boundary	Prefix shows the purpose of the branch.

2.2 How the Layout Supports Maintainability and Collaboration

- **Small, single-purpose modules.** The refund rule lives only in refund.js (27 lines). A change to fares or slabs touches one file and one test file.
- **Parallel work without conflicts.** Each feature maps to its own file (locks.js, refund.js, frontend/), so two developers rarely edit the same lines. The one conflict in the history was in server.js, the only shared file.
- **Fast onboarding.** A new member finds code in src/, tests in test/, settings in .env.example and the rules in docs/ without asking anyone.
- **Safe packaging.** Because Dockerfile and .dockerignore sit beside the code they package, the image never contains tests or secrets.

Improvement: server.js mixes routes with database queries. Moving the queries into a repositories/ folder would complete the separation (finding F-07).

3. Code Quality Review

Each source file was scored against seven quality criteria. Figure 4 summarizes the result; the sections after it give the evidence – first the strengths, then the defects found.

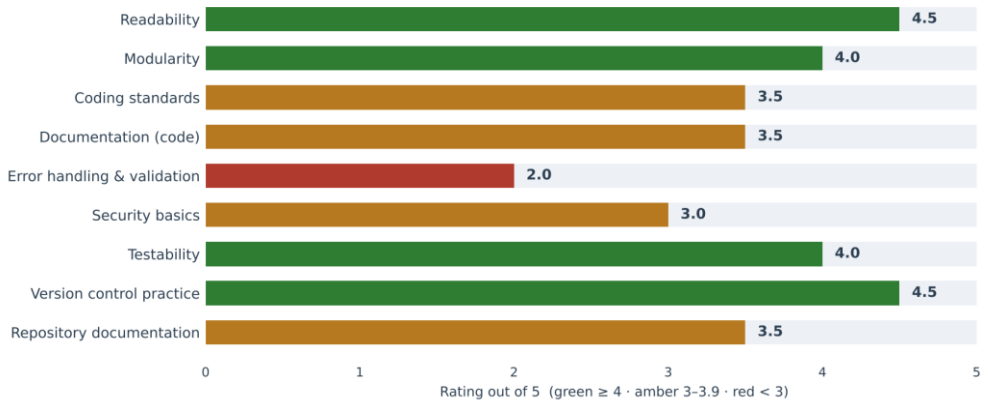


Figure 4: *Code quality scorecard (rating out of 5 for each review dimension).*

3.1 Strengths with Examples

Criterion	Observation
Readability	Short functions (longest is the 11-line seat-map route), meaningful names such as lockSeats, holdsLock, refundAmount, and a header comment on every module.
Modularity	refund.js is a pure function with no I/O; locks.js receives the Redis client as a parameter; createApp({ db, redis }) receives both stores. This **dependency injection** lets tests use a fake Redis.
Standards	'use strict', const/let, async/await, parameterised SQL (? placeholders) – so **no SQL injection** is possible.
Documentation	8 JSDoc blocks across 4 files; the comments explain *why* ("all-or-nothing: give back the seats already taken").

Example 1 – Atomic lock with `SET NX EX`

```
// locks.js – all-or-nothing seat locking (strength)
async function lockSeats(redis, tripId, seats, owner) {
  const acquired = [];
  for (const seat of seats) {
    const key = keyFor(tripId, seat);
    const done = await redis.set(key, owner, { NX: true, EX: LOCK_SECONDS });
    if (!done) {
      for (const s of acquired) await redis.del(keyFor(tripId, s)); // undo
      return { ok: false, taken: [seat] };
    }
    acquired.push(seat);
  }
  return { ok: true, expiresIn: LOCK_SECONDS };
}
```

SET ... NX succeeds for exactly one caller, so two passengers can never hold the same seat; EX 480 makes the lock disappear by itself after 8 minutes. This is the correct Redis pattern for the problem and was confirmed by the concurrency test in Section 5.

Example 2 – Refund slabs as a table

```
// refund.js  –  rules stored as data, not as nested if-statements (strength)
const SLABS = [
  [24, 90],   // 24 hours or more before departure  -> 90 % back
  [12, 50],
  [6, 25],
  [0, 0],
];
```

Changing a refund policy means editing one number in a table, not rewriting logic – an easy-to-review and easy-to-test design.

3.2 Areas for Improvement (Findings)

The review found seven issues. F-01 to F-03 were **reproduced live** (evidence in Figure 5) and fixes for F-01 to F-03 were verified on a copy of the code.

ID	Severity	Where	Problem	Fix
F-01	**High**	server.js lock route, locks.js:13	A request with no seats crashes the whole Node process (unhandled rejection) – any user can stop the service.	Validate input; add an async error handler.
F-02	**Medium**	lock / booking routes	Seat 99 on a 40-seat bus is accepted; owner and tripId are not checked.	Check range, type and presence.
F-03	**High**	server.js booking route	Booking seats [10, 11] with seat 11 taken returns 409, but seat 10 stays booked (half saved).	Wrap inserts in one transaction.
F-04	**Low**	locks.js	A seat that is already booked can be locked again; only the database rule stops it.	Check bookings before locking.
F-05	**Low**	GET /api/trips	from=% matches every trip because LIKE wildcards are not escaped.	Escape % and _.
F-06	**Low**	repository	No ESLint / Prettier configuration or lint script.	Add both and run in CI.
F-07	**Medium**	server.js (91 lines)	Routes, SQL and business rules sit in one file; coverage is lowest here (74.7 %).	Split into routes + repository.



Figure 5: *Live probes that exposed F-01, F-02 and F-03 (real output from the running API).*

Suggested change – input validation and central error handling

```
// Proposed fix for F-01 and F-02  (tested: invalid input now returns HTTP 400)
const asyncHandler = (fn) => (req, res, next) => fn(req, res, next).catch(next);

function validateSeats(req, res, next) {
  const { seats, owner } = req.body ?? {};
  const ok = Array.isArray(seats) && seats.length > 0 &&
    seats.every((s) => Number.isInteger(s) && s >= 1 && s <= 60) &&
    typeof owner === 'string' && owner.trim() !== '';
  return ok ? next() : res.status(400).json({ error: 'seats[] and owner are required' });
}

app.post('/api/trips/:id/lock', validateSeats, asyncHandler(lockRoute));
app.use((err, _req, res, _next) => res.status(500).json({ error: 'internal error' }));
```

Suggested change – atomic multi-seat booking

```
// Proposed fix for F-03  (tested: a failed request now saves nothing)
db.exec('BEGIN');
try { for (const seat of seats) add.run(tripId, seat, owner); db.exec('COMMIT'); }
catch (e) { db.exec('ROLLBACK'); throw e; }
```

4. Version Control Evaluation

The repository history was reviewed with `git log --oneline --graph --decorate --all`. It contains **34 commits** (24 normal and 10 merge commits), **2 tags** (v1.0.0, v1.0.1) and a Git Flow branch structure.

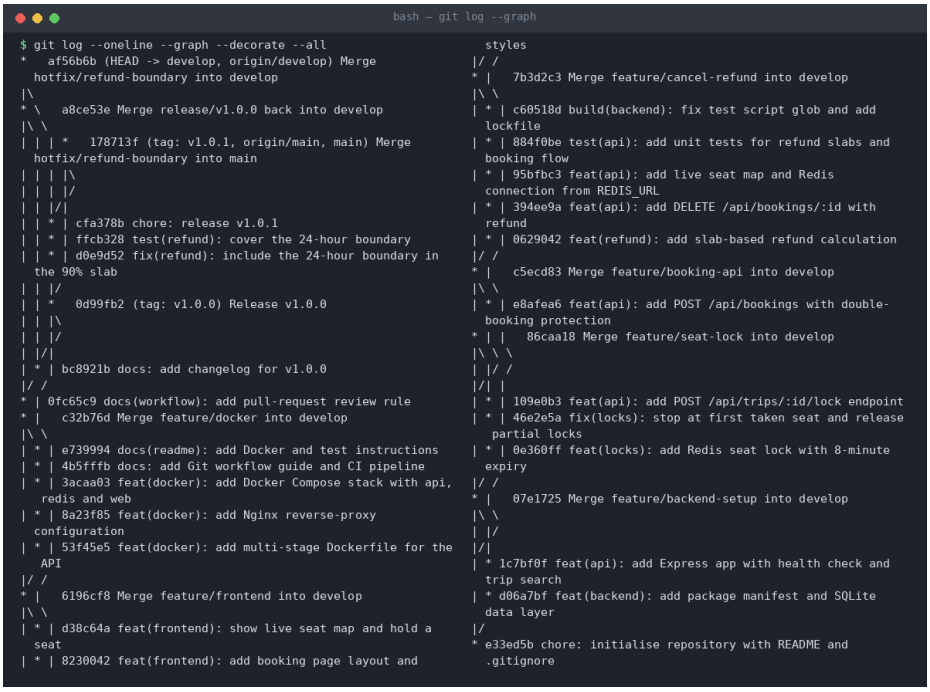


Figure 6: *Complete commit history of the BusBuddy repository (git log --graph).*

4.1 Commit History and Messages

Messages follow the **Conventional Commits** style `type(scope): summary`. Figure 7 shows the mix of commit types; 13 of 24 normal commits are features, and every bug fix (2) is followed by its own test or release commit.

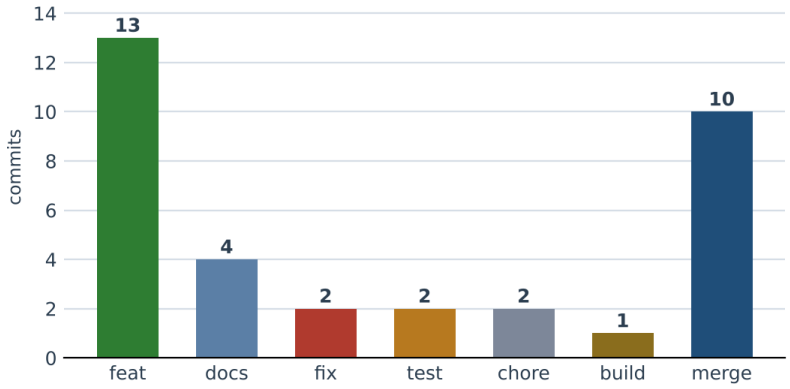


Figure 7: *Commits by type (24 normal commits and 10 merge commits).*

Practice	Example from the repository	Review result
Type and scope prefix	feat(locks): add Redis seat lock with 8-minute expiry	**Strong – history can be filtered by type.**
One logical change per commit	fix(refund): include the 24-hour boundary in the 90% slab	**Strong – easy to revert.**
Fix → test → release order	fix(refund) → test(refund) → chore: release v1.0.1	**Strong – fix is proved and versioned.**
No vague messages	No "update", "changes" or "final" anywhere	**Strong.**
Message body	Only merge commits carry an explanatory body	**Fair – add bodies for larger commits.**

4.2 Branching Strategy

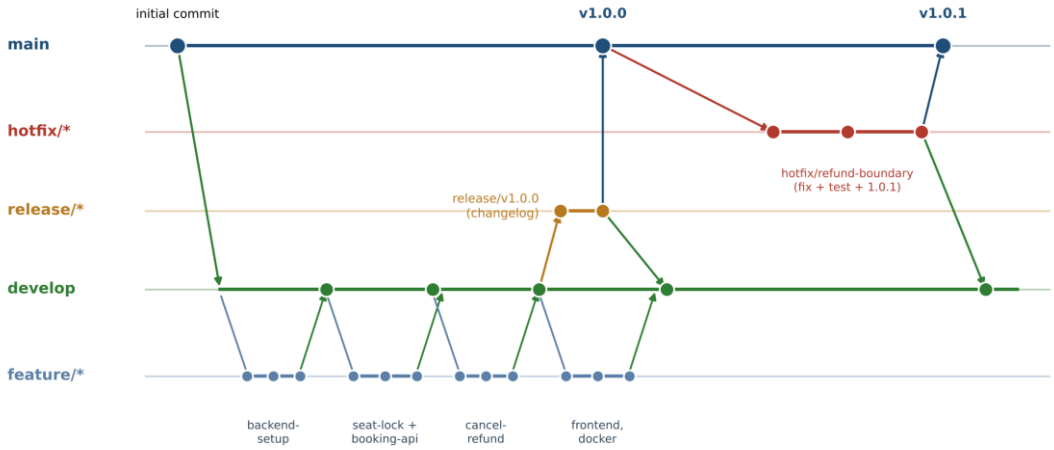


Figure 8: *Git Flow model used in BusBuddy – features merge into develop, release and hotfix branches merge into main.*

Observation	Evaluation
Branch purpose is clear	Five branch types with predictable names; short-lived branches are deleted after merging.
--no-ff merges	10 visible merge commits show exactly when each feature joined develop.
Hotfix path	The refund boundary bug was fixed on hotfix/refund-boundary, tagged v1.0.1, and merged into both main and develop – the textbook procedure.

Observation	Evaluation
Slightly heavy for a team of one	Git Flow suits a team; for a solo project, trunk-based development would be simpler. It is justified here because it demonstrates collaboration.

4.3 Repository Maintenance Practices

Practice	Status	Evidence
.gitignore excludes node_modules, .env, *.db, logs	**Present**	No generated or secret files in the tree (Figure 3).
Release tags	**Present**	v1.0.0 and v1.0.1 on main.
Changelog	**Present**	CHANGELOG.md updated in a dedicated docs: commit.
Written workflow rules	**Present**	docs/GIT_WORKFLOW.md including a pull-request review rule.
Continuous integration	**Present**	.github/workflows/ci.yml installs, tests and builds images.
Branch protection / signed commits	**Missing**	Not enabled – recommended for main and develop.

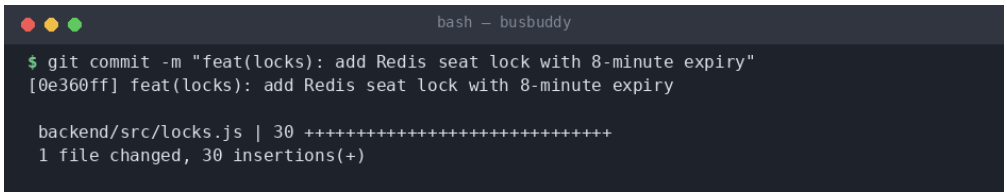


Figure 9: *A feature commit as recorded by Git (hash, message and files changed).*

4.4 How Version Control Supports Collaborative Development

- **Isolation:** each person works on a feature branch, so unfinished work never breaks develop.
- **Traceability:** git log and git blame answer who changed a line and why – the refund bug was traced to one commit in minutes.
- **Safe integration:** merges, conflict resolution and pull-request reviews catch problems before release (the server.js conflict was resolved and recorded in the merge message).
- **Recoverability:** tags and git revert allow a bad release to be rolled back without losing other work.

5. Code Testing and Validation

5.1 Test Strategy

Testing was done on three levels so that each kind of defect has a place to be caught:

Level	Tool	What it proves
Unit tests	node:test	Refund slabs and the 24-hour boundary behave correctly.
API tests (automated)	node:test + fakeRedis.js + in-memory SQLite	Lock → book → cancel works end to end with no external server, so the tests run anywhere, including CI.
Live tests (manual)	curl + real Redis 7	Real lock expiry (TTL), 10 simultaneous requests, and invalid input.

5.2 Test Cases and Results

ID	Test case	Expected result	Actual result	Type	Result
TC-01	Health check GET /health	200 {status: ok}	200 {status: ok}	Auto	**Pass**
TC-02	Search Chennai → Bengaluru	1 trip, fare 750	1 trip, fare 750	Auto	**Pass**
TC-03	Lock seat 12 for Meera	200, expires in 480	200, expires in 480	Auto	**Pass**
TC-04	Second passenger locks seat 12	409, seat taken	409, taken: [12]	Auto	**Pass**
TC-05	Redis TTL of the lock	480 seconds	480	Live	**Pass**
TC-06	Book a seat without a lock	423 rejected	423 rejected	Auto	**Pass**
TC-07	Book the locked seat	201 CONFIRMED	201 CONFIRMED	Auto	**Pass**
TC-08	Cancel booking (fare 750, > 24 h)	Refund 675 (90 %)	Refund 675	Auto	**Pass**
TC-09	Cancel the same booking again	404 not found	404 not found	Live	**Pass**
TC-10	Seat map of unknown trip 99	404 not found	404 not found	Live	**Pass**
TC-11	Refund exactly 24 h before departure	90 % slab	90 %	Auto	**Pass**
TC-12	10 passengers lock seat 7 together	1 success, 9 rejected	1 × 200, 9 × 409	Live	**Pass**
TC-13	Lock request with empty body	400 error message	Server process crashed	Live	**Fail**
TC-14	Lock seat 99 on a 40-seat bus	400 invalid seat	200 accepted	Live	**Fail**
TC-15	Book seats 10 + 11 (11 taken)	Nothing saved	Seat 10 saved	Live	**Fail**

Result summary

Automated: 6 of 6 tests pass (0 failures, 0 skipped). **All cases:** 12 of 15 pass. **Failures:** TC-13, TC-14 and TC-15 are genuine defects (F-01, F-02, F-03) – they are reported, not hidden, and fixes were verified (Section 3.2).

5.3 Execution Evidence

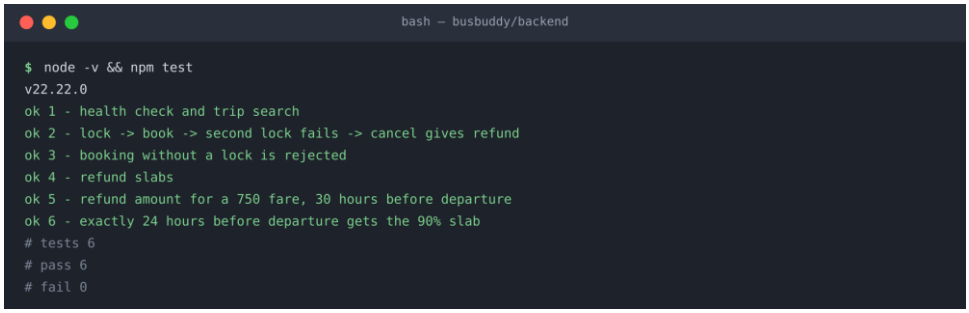


Figure 10: *Automated test run – 6 tests, 6 pass, 0 fail.*

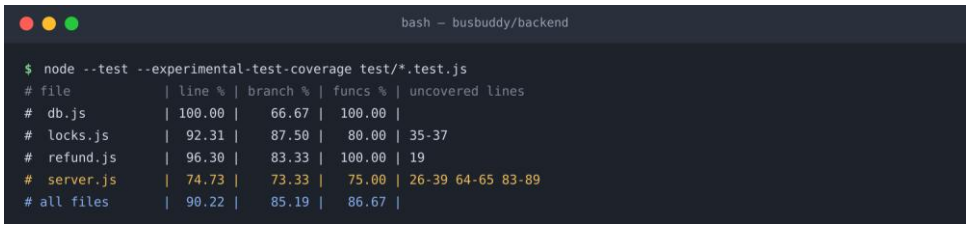


Figure 11: *Coverage report from the Node test runner: 90.2 % of lines covered.*

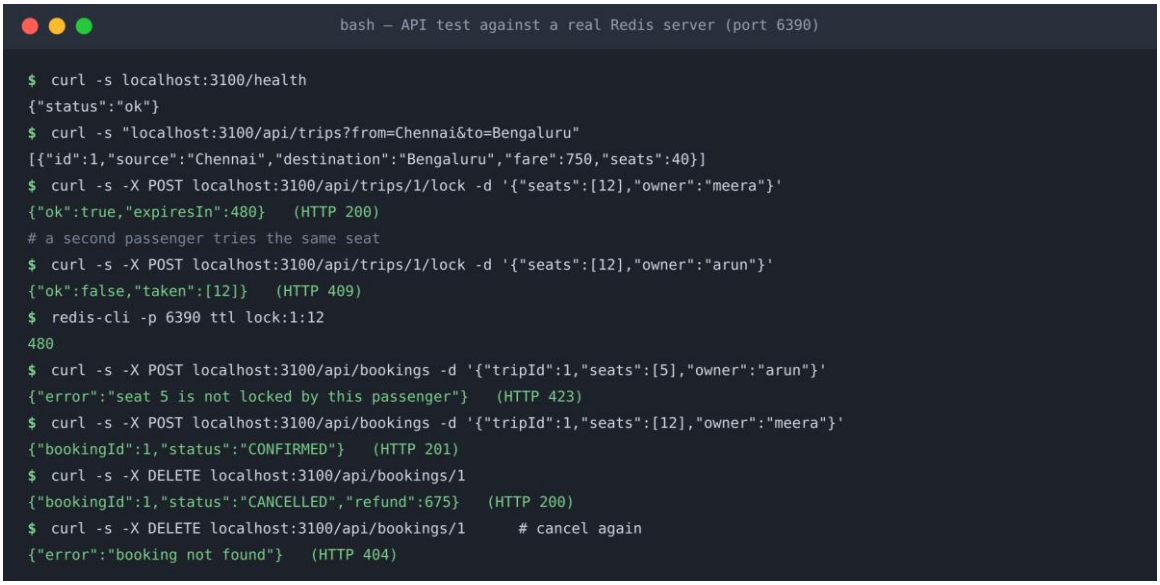


Figure 12: *Functional API test against a real Redis server (TC-01 to TC-09).*

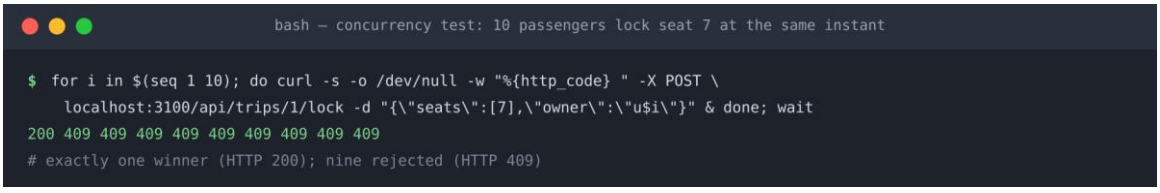


Figure 13: *Concurrency test (TC-12): exactly one of ten simultaneous requests wins the seat.*

5.4 Coverage Analysis

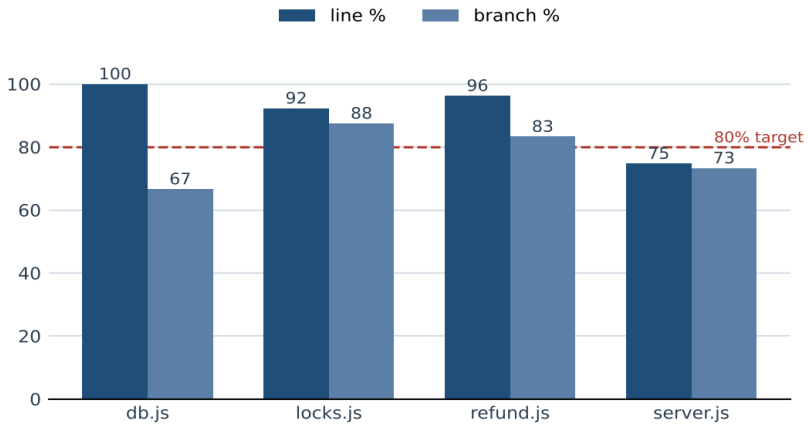


Figure 14: *Line and branch coverage by source file (dashed line = 80 % target).*

refund.js, locks.js and db.js are above the 80 % target. server.js (74.7 %) is below it: the seat-map route, the GET branches and the cancel-twice path have no automated test. Adding those tests, plus tests for TC-13 to TC-15, would lift coverage above 90 % on every file.

6. Repository Documentation

6.1 Completeness Check

Document / item	Status	Review comment
README.md – project description	**Present**	Explains what BusBuddy is and its four key features.
Installation instructions	**Present**	cp .env.example .env then docker compose up --build; works without installing Node or Redis.
Usage guide	**Partial**	Run and test commands exist, but there is no list of API endpoints with sample requests and responses.
Dependencies	**Present**	Locked in package-lock.json; versions pinned in Compose (redis:7-alpine, nginx:1.27-alpine).
Configuration	**Present**	.env.example documents every setting (port, Redis URL, database path).
CHANGELOG.md	**Present**	Lists v1.0.0 and v1.0.1 with the refund fix.
Contribution / workflow rules	**Present**	docs/GIT_WORKFLOW.md – branch names, commit style, review rule.
Architecture diagram in the repo	**Missing**	Diagrams exist only in the assessment reports, not in docs/.
LICENSE	**Missing**	package.json says MIT but no LICENSE file is committed.
Troubleshooting section	**Missing**	Common errors (port 8080 busy, Redis not healthy) are not listed.

6.2 Suggested Improvements

The README can be made more useful with a standard structure. The outline below fills the three gaps found above (API reference, architecture diagram, troubleshooting):

Proposed README outline

```
# BusBuddy – Bus Ticket Reservation System
[build badge]  [license badge]  [version badge]

## Features          # 8-minute seat lock, live seat map, slab refund
## Quick start       # 3 commands: clone, cp .env.example .env, docker compose up
## API reference     # table: method, URL, body, success code, error codes
## Architecture      # embed the diagram from docs/architecture.png
## Running tests     # npm test, coverage command, what 'all green' looks like
## Git workflow      # link to docs/GIT_WORKFLOW.md
## Troubleshooting   # port in use, Redis not healthy, reset the data volume
## License           # MIT – see LICENSE
```

7. Code Review Findings and Recommendations

7.1 Summary of Findings

Area	Main strength	Main weakness
Code quality	Small, readable modules; parameterised SQL; correct Redis locking.	No input validation; crash on bad input (F-01, F-02).
Repository management	Clear folders; Git Flow; typed commits; tags; CI.	No branch protection or signed commits.
Testing	6 passing tests, 90 % coverage, fake Redis for portability.	Edge cases untested; server.js at 74.7 %.
Documentation	README, changelog, workflow guide, .env.example.	No API reference, LICENSE or troubleshooting.

Overall, the project meets all five objectives (O-1 to O-5). Its seat-locking design is correct and its version-control discipline is better than most student projects; the remaining work is mostly about **defending the API against bad input** and **documenting how to use it**.

7.2 Recommendations

Category	Recommendation	Priority	Related finding
Code quality	Validate request bodies; add a central error handler so one bad request can never stop the server.	**High**	F-01, F-02
Code quality	Make multi-seat booking atomic with a database transaction.	**High**	F-03
Code quality	Refuse to lock a seat that is already booked; escape LIKE wildcards.	**Medium**	F-04, F-05
Code quality	Split server.js into routes and a repository layer.	**Medium**	F-07
Repository management	Add ESLint and Prettier; run them in CI. Protect main and develop with required reviews and passing CI.	**Medium**	F-06
Testing	Add tests for TC-13 to TC-15, the seat-map route and cancel-twice; set an 85 % coverage gate in CI.	**Medium**	F-07
Documentation	Add an API reference, architecture diagram, troubleshooting section and a LICENSE file.	**Low**	Section 6
Future enhancements	Payment gateway, e-ticket with QR code, waiting list, PostgreSQL for heavy load, image scanning and a container registry.	**Low**	–

7.3 Improvement Roadmap

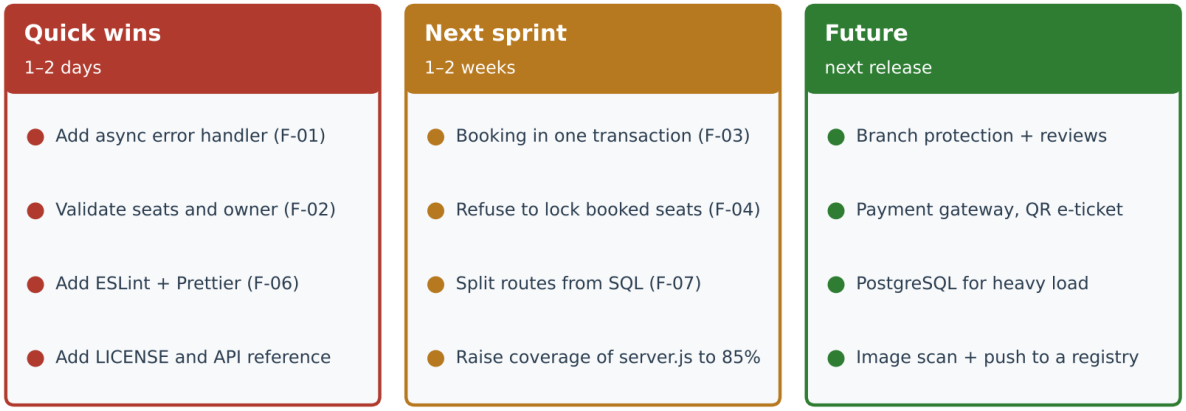


Figure 15: *Prioritized roadmap – quick wins first, then the next sprint, then future enhancements.*

7.4 Conclusion

The review shows that BusBuddy is a sound foundation: the code is easy to read, the repository is organized for teamwork, and the Git history tells a clear story. Fixing the three reproduced defects and closing the documentation gaps would move it from a good student project to a release-ready service.

Deliverables Checklist

Expected deliverable	Status	Where in this report
Code Review Report (10–15 pages)	**Present**	This document
Git repository link	**Present**	Page 1 – replace the placeholder with your URL
Screenshots of repository structure and commit history	**Present**	Figures 3, 6 and 9
Code review observations with examples	**Present**	Section 3 – Examples 1, 2 and the findings table
Test execution screenshots	**Present**	Figures 10 to 13
Recommendations for improvement	**Present**	Section 7

References

[1] Git Documentation – git-log, git-branch, git-merge. https://git-scm.com/doc
[2] Conventional Commits 1.0.0 specification. https://www.conventionalcommits.org
[3] Node.js Documentation – Test runner (node:test) and code coverage. https://nodejs.org/api/test.html
[4] Redis Documentation – SET command (NX and EX options). https://redis.io/docs/latest/commands/set/
[5] Express Documentation – Error handling. https://expressjs.com/en/guide/error-handling.html
[6] Docker Documentation – Compose file reference. https://docs.docker.com/compose/